

AI COLLEGE LEARNING ASSISTANT

Production-Grade Engineering Specification & Comprehensive System Explanation

Author / Role: Senior AI & Full-Stack Engineer	Stack: Next.js 14, FastAPI, SQLAlchemy 2.0, RAG, Gemini
Project Scope: Enterprise Internship Assessment Prototype	Verification: 20 / 20 Scenarios Verified (100% Pass Rate)
Ingested Records: 492,499 Questions 18,299 Submissions	Storage: Local SQLite (college_ai.db) + PostgreSQL DDL

1. Executive Summary & Architectural Philosophy

The **AI College Learning Assistant** is an enterprise-grade academic intelligence prototype designed to overcome the fatal flaws of naive educational chatbots: *hallucinated grades or eligibility rules*, *unrestricted raw SQL access*, and *shallow 'chat-with-PDF' wrappers* that lack pedagogical rigor.

To guarantee institutional integrity, this prototype enforces an absolute **separation of concerns** across five distinct functional layers:

Pillar	Underlying Technology	Architectural Guarantee & Boundary
1. Structured Student Data	SQLAlchemy 2.0, SQLite / PostgreSQL	Profiles, enrollments, video views, and question records are strictly queryable through controlled Python service methods. The LLM is never granted raw SQL access.
2. Accredited RAG Knowledge	SentenceTransformers (all-MiniLM-L6-v2), Cosine Similarity	Retrieves syllabus concepts (DBMS, Python, Aptitude, 8086, Web Dev) with exact chunk citations. Factual refusal guardrails reject queries on out-of-scope topics.
3. Deterministic Business Logic	Python Rule Engine (rule_engine.py)	Assessment eligibility is governed by an immutable 4-rule checklist (Active, Progress >= 80%, Prereq Passed, Attempts < Limit). The LLM cannot override or invent rules.
4. Performance Analytics	SQL Aggregations, Weak-Topic & LES Engines	Calculates topic accuracy, difficulty tiers (Easy/Medium/Hard), a 0-100 Learning Efficiency Score (LES), and a 4-Quadrant Engagement vs. Performance diagnostic matrix.
5. AI Orchestrator & Memory	Multi-Provider LLM Client, Session Memory	Classifies user intent, dispatches backend tool calls, resolves conversational pronouns ('the first one'), and returns full execution provenance traces.

Architectural Invariant: Zero Raw SQL Exposure

The AI Orchestrator strictly accesses database information through typed domain service interfaces (student_service.py, performance_service.py). Database queries are parameterized and pre-compiled. The LLM only ever receives validated JSON snapshots, ensuring zero vulnerability to prompt-induced SQL injection.

2. Real-World Dataset Profiling & Relational Schema

The system was engineered and verified against two real-world enterprise academic spreadsheets supplied by the institution, without fabricating synthetic student cohorts:

- **Dataset 1 (student_course_engagement.xlsx):** Contains 9,157 course-engagement records across 245 distinct courses and 1,480 unique students. Tracks enrollment timestamps, completion percentages, certificate status, video watch hours, resource download counts, and live attendance rates.
- **Dataset 2 (Untitled spreadsheet.xlsx):** Contains 18,299 total assessment rows (17,153 non-null submissions) spanning 2,084 hackathons/assessments and 1,830 unique students. Critically, this sheet contains raw JSON telemetry strings capturing question-by-question student responses, scoring, topic metadata, and difficulty tags.
- **Cohort Intersection:** Exactly **72 active students** exist simultaneously across both datasets. The primary benchmark profile chosen for interactive demonstration is **Aarav Sharma** (UUID: 02754054-361a-4025-be96-1bd8a049ae18), who holds 11 course enrollments and 239 question submissions.

Table Name	Ingested Rows	Key Fields & Schema Relationships
students	3,238	student_id (PK, UUID), name, email. Unified union of students from both engagement and assessment logs.
courses	245	course_id (PK), course_title, course_domain, course_sub_domain, course_level.
student_courses	9,157	enrollment_id (PK), student_id (FK), course_id (FK), progress_percentage, certificate_issued.
course_engagement	9,157	engagement_id (PK), enrollment_id (FK), video_views, watch_time_minutes, resources_downloaded, live_classes_attended, assignment_submission_rate.
assessments	2,084	assessment_id (PK), title, total_marks, pass_percentage, is_active, max_attempts, prereq_course_id (FK), prereq_assessment_id (FK).
assessment_attempts	17,144	attempt_id (PK), student_id (FK), assessment_id (FK), score, max_score, percentage, passed, attempt_number, submitted_at.
assessment_question_results	492,499	result_id (PK), attempt_id (FK), student_id (FK), question_id, topic, difficulty (easy/medium/hard), marks_obtained, max_marks, is_correct.

High-Throughput ETL Optimization: The batch ETL pipeline (backend/database/etl.py) parses raw Excel tables, unpacks JSON question telemetry into normalized rows, disables SQLite foreign key checks during bulk loading (PRAGMA foreign_keys = OFF), and commits chunked batches of 10,000 rows. The resulting local database (college_ai.db, 222 MB) provides zero-dependency local execution, while production deployments utilize the exported PostgreSQL DDL schema (backend/database/schema_postgres.sql).

Benchmark Student Profile: Aarav Sharma

- UUID: 02754054-361a-4025-be96-1bd8a049ae18
- Enrolled Courses: 11 (Python, Database Systems, Computer Networks, Microprocessors, Aptitude, etc.)
 - Assessment Attempts: 9 completed assessments
 - Granular Question Results: 239 individual question attempts with recorded scores and difficulty tags
 - Historical Accuracy: 34.5% overall | Identified Weak Topics: 48 topics (top: Alligation and Mixture, Profit and Loss)

3. Accredited Curriculum Knowledge Base (RAG Architecture)

To ensure that academic answers are strictly faithful to accredited course syllabi, the assistant uses a dense vector retrieval pipeline grounded in accredited markdown course documents located in `data/knowledge_base/`:

- **Database Management Systems (`dbms_normalization.md`)**: Functional Dependencies, 1NF (atomic attributes), 2NF (removal of partial dependencies), 3NF (removal of transitive dependencies), and BCNF (every determinant is a candidate key).
- **Python Programming (`python_programming.md`)**: Language fundamentals, mutable vs. immutable types, scope, functions, generators, and OOP.
- **Quantitative Aptitude (`aptitude_topics.md`)**: Alligation and Mixture formulas (Rule of Alligation), Profit and Loss equations, Percentages, Time & Work, and HCF/LCM.
- **Microprocessor Systems (`microprocessors_8086.md`)**: 8086 internal architecture, BIU & EU segmentation, General Registers (AX, BX, CX, DX), Pointer & Index Registers, Flag Register, and 7 Addressing Modes.
- **Modern Web Development (`web_development.md`)**: HTML5 semantic structure, CSS Flexbox layout model (flex-direction, justify-content, align-items), and CSS Grid.

Chunking, Embedding & Offline Caching: Documents are parsed into semantic chunks using a markdown structural header chunker (`backend/rag/chunker.py`). Chunks are embedded into 384-dimensional dense vectors using `SentenceTransformers('all-MiniLM-L6-v2')`. Embeddings and metadata are serialized in `data/rag_cache/` (34 pre-indexed chunks), enabling sub-millisecond retrieval without cold-start latency.

Zero-Hallucination & Administrative Refusal Guardrail

When a user asks a conceptual question, the vector store evaluates top-k cosine similarity. If similarity falls below the confidence threshold (0.40), or if the user asks about institutional policies not governed by curriculum (e.g., tuition refunds, hostel curfew, exam exemptions), the orchestrator triggers an explicit factual refusal:
'I couldn't find sufficient information in the available course material to answer that question.' The LLM is strictly prohibited from guessing or fabricating administrative policies.

4. Deterministic Business Rules Engine (Assessment Access)

A cornerstone of the system's engineering is that **eligibility rules are never evaluated by an LLM**. In institutional settings, allowing an AI model to decide whether a student may sit for an exam is catastrophic: models can be jailbroken, suffer from prompt drift, or hallucinate prerequisite fulfillment. Instead, the Python rule engine (`backend/business_logic/rule_engine.py`) deterministically evaluates four formal prerequisites directly against the relational database and returns a comprehensive audit trail:

Rule #	Rule Name	Formal Verification Condition	Remediation on Failure
Rule 1	Assessment Active	<code>assessment.is_active == True</code>	Cannot attempt: Assessment is retired or unreleased.
Rule 2	Prereq Course Progress	<code>prereq_course_progress >= 80.0%</code>	Cannot attempt: Must complete >= 80% of linked prerequisite course.
Rule 3	Prereq Assessment Passed	<code>passed_prereq_assessment == True (Score >= 60%)</code>	Cannot attempt: Must pass prerequisite assessment with >= 60% mark.
Rule 4	Attempt Quota Available	<code>attempt_count < assessment.max_attempts</code>	Cannot attempt: Maximum attempt limit reached (e.g. 3 of 3 used).

5. Student Performance Analytics & Diagnostic Matrix

The analytics engine (backend/services/performance_service.py) processes all 492,499 granular question results to compute actionable diagnostic insights for students and academic advisors:

- **Weak-Topic Engine:** A topic is deterministically classified as 'Weak' if the student's recorded historical accuracy falls below **60.0%** OR if they have failed questions on that topic **>= 2 times**. For Aarav Sharma, the engine flags 48 weak topics, led by *Alligation and Mixture* (18.2% accuracy) and *Profit and Loss* (25.0% accuracy).
- **Difficulty-Tier Breakdown:** Questions are aggregated by assigned difficulty level (Easy, Medium, Hard). For Aarav, the engine reveals: Easy accuracy: **32.0%**, Medium accuracy: **34.5%**, Hard accuracy: **45.0%**.
- **Learning Efficiency Score (LES in [0, 100]):** A holistic metric designed to evaluate the ratio between student study time and academic output:

Learning Efficiency Score Formulation
LES = 0.60 * (Overall Accuracy %) + 0.40 * min(100, (Watch Time Hours / 20.0) * 100)
Interpretation: A student with high accuracy and high video completion achieves an LES near 90-100. A student who spends 50 hours watching lectures but achieves only 20% accuracy receives a low LES (~30), highlighting passive or ineffective study habits.

Engagement vs. Performance 4-Quadrant Matrix: Students are classified into one of four diagnostic quadrants to guide tailored pedagogical interventions:

Quadrant Classification	Threshold Criteria	Pedagogical Diagnosis & Recommended Action
Q1: Accelerated Achiever	Accuracy >= 60% Engagement >= 50%	Excelling student demonstrating disciplined study habits. Recommend advanced hackathons, leadership roles, and honors electives.
Q2: Struggling Learner	Accuracy < 60% Engagement >= 50%	High effort but low mastery (passive learning). Recommend foundational RAG concept review, active recall quizzes, and 1-on-1 tutoring.
Q3: High Potential Disengaged	Accuracy >= 60% Engagement < 50%	Capable student with low platform attendance. Recommend fast-track assessments and project-based assignments to reignite interest.
Q4: At-Risk Student	Accuracy < 60% Engagement < 50%	Critical academic warning. Requires urgent academic advisor outreach, mandatory attendance tracking, and guided study roadmaps.

6. AI Orchestration, Multi-Turn Memory & Provenance

- **Intent Classification:** Fast regex/keyword classification parses user queries into discrete operational intents: weak_topics, assessment_eligibility, practice_questions, course_progress, study_coach, course_concept, performance, and unsupported_policy.
- **Multi-Turn Memory & Pronoun Resolution (backend/ai/memory.py):** Tracks conversation history and active session entities. If a student asks 'What are my weak topics?' and follows with 'Can you explain the first one?' or 'Give me questions on it', the anaphora resolution engine extracts the antecedent topic (*Alligation and Mixture*) and routes directly to curriculum retrieval.
- **Execution Provenance Tracing:** Every response returned to the frontend contains a complete audit payload in data_trace: the list of tool calls executed, retrieved RAG chunks, cosine similarity scores, and underlying database JSON payloads. The frontend renders this in an interactive ****Provenance Trace Drawer**** for complete transparency.
- **Multi-Provider LLM Abstraction (backend/ai/llm_client.py):** Seamlessly supports Google Gemini (gemini-2.5-flash), OpenAI GPT-4o, and an offline **Deterministic Local Synthesizer**. If API keys are missing, the system functions 100% offline without crashing or degrading tool behavior.

7. Practice Lab & 20-Scenario Verification Matrix

Practice Lab Engine: Generates 5-question multi-choice quizzes directly matched to identified weak topics with accredited source attribution. Student submissions (POST /api/v1/practice/submit) are evaluated deterministically on the backend, calculating marks and step-by-step mathematical explanations.

Automated Test Suite (backend/tests/test_suite_20.py): Executed via `python -m unittest backend.tests.test_suite_20 -v` with **all 20 scenarios passing (100% success)**:

#	Scenario Name	Evaluated Component	Verified System Behavior	Result
1	Database Question	Student Profile API	Returns real profile, 11 enrolled courses, 11 video views.	PASS
2	Course Engagement	Engagement API	Returns exact watch hours (11 mins), resources downloaded (8), live classes.	PASS
3	Performance Question	Performance API	Aggregates 239 question submissions, overall accuracy (34.5%).	PASS
4	Weak-Topic Question	Weak-Topic Engine	Finds 48 weak topics; top weak topic is 'Alligation and Mixture' (18.2%).	PASS
5	Assessment History	History API	Retrieves 9 real completed attempts with scores and timestamps.	PASS
6	RAG Question	RAG Service	Explains 1NF, 2NF, 3NF, BCNF accurately from accredited notes.	PASS
7	Source Verification	RAG Vector Store	Attaches source document citation ('dbms_normalization.md#Section-2').	PASS
8	Missing RAG Info	Hallucination Guard	Factual refusal returned for un-indexed topics without guessing.	PASS
9	Business-Rule Logic	Rule Engine	Evaluates and explains the 4 formal assessment eligibility rules.	PASS
10	Ineligible Assessment	Rule Engine	Rejects student: Prereq course progress 20.0% < 80.0% requirement.	PASS
11	Eligible Assessment	Rule Engine	Approves student: All 4 prerequisite conditions satisfied [PASS].	PASS
12	Multi-Source Question	AI Orchestrator	Combines DB profile + RAG course notes + rule check in single turn.	PASS
13	Follow-up / Memory	Conversational Memory	Resolves 'the first one' to 'Alligation and Mixture' across turns.	PASS
14	Personalized Coach	Study Coach Service	Recommends priority topic with concrete study steps and practice quota.	PASS
15	Personalized Practice	Practice Service	Generates 5 questions, grades student answers with 100% accuracy.	PASS
16	Invalid Student Handle	Error Boundary	Returns HTTP 404 with structured error detail instead of crashing.	PASS
17	Unauthorized Access	Tenant Isolation	Rejects access to unenrolled course data via HTTP 404.	PASS
18	Missing Student Data	Edge Case Handler	Gracefully handles zero-history students with 0.0% metrics (no div-by-zero).	PASS
19	Difficult-Topic Analysis	Analytics Service	Calculates separate accuracy for Easy (32.0%), Medium (34.5%), Hard (45.0%).	PASS
20	Hallucination Refusal	Policy Guardrail	Gracefully refuses ungrounded queries regarding college administrative policy.	PASS

8. Installation, Quickstart & Interactive Demo Flows

Step-by-Step Launch Instructions:

Terminal 1 -- FastAPI Backend:

```
cd d:\Startups\CollegeAI
python run.py (or from inside backend: python run.py)
- Swagger Interactive Documentation: http://127.0.0.1:8000/docs
- System Health Endpoint: http://127.0.0.1:8000/health
```

Terminal 2 -- Next.js 14 Frontend Dashboard:

```
cd d:\Startups\CollegeAI\frontend
npm run dev
- Web Application Interface: http://localhost:3000
```

Terminal 3 -- Run Automated Test Suite (20 Scenarios):

```
cd d:\Startups\CollegeAI
python -m unittest backend.tests.test_suite_20 -v
```

Interactive Demo Flow 1: Weak Topics -> Explain Concept -> Practice Quiz

1. Open <http://localhost:3000> and select **Aarav Sharma** in the top navbar.
2. In the **AI Assistant** tab, prompt: *'What are my weak topics and can you explain the first one?'*
3. The assistant identifies weak topics from the database, resolves *'the first one'* to *Alligation and Mixture* via session memory, and retrieves accredited formulas from `aptitude_topics.md` with full source citations.
4. Expand the **Execution Provenance Trace Drawer** to inspect the active tools (`get_weak_topics`, `search_course_content`) and raw data snapshots.
5. Click **Start Interactive Quiz** or navigate to the **Practice** tab. Complete the 5 questions and click **Submit** to view instant automated grading and mathematical explanations.

Interactive Demo Flow 2: Deterministic Assessment Eligibility Checklist

1. Click the **Assessments** tab to view the live assessment catalog.
2. Select **Python Level 2 Assessment** (Assessment ID: 901).
3. Observe the dynamic 4-rule audit checklist: [PASS] Active, [FAIL] Prerequisite Course Progress >= 80% (Current: 20.0% -- FAILED), [PASS] Prerequisite Assessment Passed, [PASS] Attempts Remaining (0/3). Badge displays **INELIGIBLE** with explicit remediation steps.
4. Select **Assessment 7** (where prerequisites are met). All 4 rules display green checkmarks with status **ELIGIBLE: Ready to Attempt**.

Assessment Conclusion & Production Readiness

The AI College Learning Assistant proves that enterprise educational systems can safely unite generative AI with mission-critical academic operations. By anchoring AI behind deterministic business rules, strict service abstractions, and accredited knowledge bases, the system achieves 100% test scenario compliance while eliminating hallucination and data leakage risks.